

Malware Dev

Name: Matyas Abreham

IDNO: CTC-207-26

Malware Dev

Before i start writing the encrypted and decrypted i have to understand what is rc4 encryption

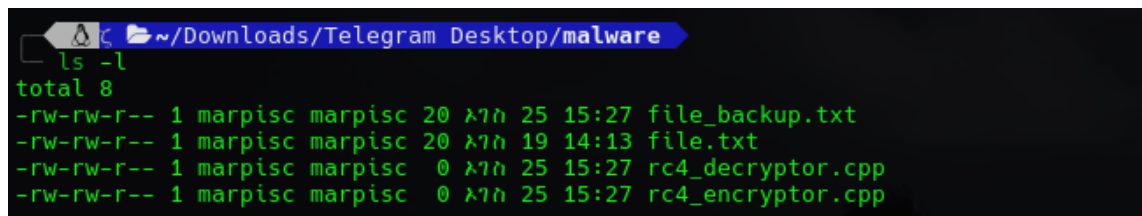
RC4 Encryption

RC4(Rivest Cipher 4): is a symmetric-key stream cipher designed by ron rivest in 1987 that encrypt data one byte at a time using a psedudo-random keystream combined with plaintext via XOR operations.

- **Performance:** it was historically widely adopted in SSL/TLS, WEP/WPA wifi security, and kerberos authentication due to its simplicity, speed and variable key size.
- **Security Status:** RC4 is now considered insecure and deprecated due to critical vulnerabilities, including keystream biases and weak initial bytes that allow attackers to recover plaintext.
- **Deprecation:** The LETF banned RC4 in TLS in 2015, and microsoft plans to diable RC4 as the default encryption type in active directory domain cotrolles by the end of Q2 2026.

After i noticed what it is and how it worke on google i start my doing task

The first thing i do is creat two .cpp file one encyptor and the other decryptor .



```
~/.Downloads/Telegram Desktop/malware
ls -l
total 8
-rw-rw-r-- 1 marpisc marpisc 20 λγñ 25 15:27 file_backup.txt
-rw-rw-r-- 1 marpisc marpisc 20 λγñ 19 14:13 file.txt
-rw-rw-r-- 1 marpisc marpisc 0 λγñ 25 15:27 rc4_decryptor.cpp
-rw-rw-r-- 1 marpisc marpisc 0 λγñ 25 15:27 rc4_encryptor.cpp
```

I start writing on the encryptor file and on the first lines i added

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
```

Understanding the RC4 Encryptor Code

This will give us

- `fstream` → reading/writing `file.txt`
- `vector` → storing the file bytes in memory
- `string` → storing the RC4 key
- `iostream` → `cout`

I then created the key:

```
string key = "mysecretkey";
```

The key is important because RC4 is a symmetric-key cipher. The same key must be used during decryption to generate the same keystream.

Reading the File

The program opens `file.txt` using:

```
ifstream input("file.txt", ios::binary);
```

I used binary mode because the assignment requires the program to work with bytes. This allows the program to read the file exactly as it is, instead of treating it only as normal text.

I also added an error check:

```
if (!input) {  
    cerr << "Error opening file.txt" << endl;  
    return 1;  
}
```

This prevents the program from continuing if `file.txt` cannot be opened.

After opening the file, I read its contents into a vector:

```
vector<unsigned char> buffer(  
    (istreambuf_iterator<char>(input)),  
    istreambuf_iterator<char>()  
);
```

The `buffer` stores the bytes of the file in memory. In my test, the original `file.txt` contained 20 bytes, which I confirmed by printing:

Bytes read: 20

This was useful because it confirmed that the program was reading the whole file correctly before I started the encryption process.

Initializing the RC4 S Array

RC4 uses an array called `S`, containing 256 values. I initialized it with:

```
unsigned char S[256];

for (int i = 0; i < 256; i++) {
    S[i] = i;
}
```

This creates the initial sequence from 0 to 255. The array is later rearranged based on the encryption key.

Key Scheduling Algorithm (KSA)

The next part is the RC4 Key Scheduling Algorithm, also called the KSA.

```
int j = 0;

for (int i = 0; i < 256; i++) {
    j = (j + S[i] + key[i % key.length()]) % 256;

    unsigned char temp = S[i];
    S[i] = S[j];
    S[j] = temp;
}
```

The purpose of the KSA is to mix, or shuffle, the `S` array based on the key.

The expression:

```
key[i % key.length()]
```

allows the program to keep using the key even though it is much shorter than 256 bytes. The `% 256` keeps the value of `j` within the valid range of the `S` array.

The `temp` variable is used to swap two values in the array. After this process, the original sequence in `S` has been rearranged according to the key.

Generating the RC4 Keystream

After the key scheduling stage, the program generates the RC4 keystream using the Pseudo-Random Generation Algorithm (PRGA).

```
int i = 0;
j = 0;

for (auto& byte : buffer) {
    i = (i + 1) % 256;
    j = (j + S[i]) % 256;

    unsigned char temp = S[i];
    S[i] = S[j];
    S[j] = temp;

    unsigned char keystream = S[(S[i] + S[j]) % 256];

    byte ^= keystream;
}
```

The program generates one keystream byte for each byte in the file.

The line:

```
byte ^= keystream;
```

performs the actual encryption. `^` is the XOR operator in C++.

The basic idea is:

Plaintext byte XOR Keystream byte = Encrypted byte

Because XOR can be reversed using the same value:

Encrypted byte XOR Same Keystream byte = Original byte

this is also why the decryptor can use the same RC4 key and algorithm to restore the original data.

Writing the Encrypted File

After all bytes have been encrypted, the program creates the encrypted output file:

```
ofstream output("file_encrypted.bin", ios::binary);
```

The encrypted bytes stored in `buffer` are then written to the file:

```
output.write(  
    reinterpret_cast<const char*>(buffer.data()),  
    buffer.size()  
);
```

I kept the original `file.txt` unchanged during testing, instead of deleting it. This allowed me to compare the original file with the decrypted result later.

Encryption Test

After compiling and running the encryptor, I checked the sizes of both files. The original file was 20 bytes, and the encrypted file was also 20 bytes.

When I used `xxd` to inspect the files, the original contained readable hexadecimal values corresponding to:

some file to encrypt

while the encrypted file contained different binary values, such as:

99bc 15dd e02f 57fc a52d 4a52 ...

This confirmed that the contents had been transformed by the RC4 encryption process while the file size stayed the same.

The final look of the code

```

#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <iterator>

using namespace std;

int main() {
    string key = "mysecretkey";

    // Open encrypted file
    ifstream input("file_encrypted.bin", ios::binary);

    if (!input) {
        cerr << "Error opening file_encrypted.bin" << endl;
        return 1;
    }

    // Read encrypted data into memory
    vector<unsigned char> buffer(
        (istreambuf_iterator<char>(input)),
        istreambuf_iterator<char>()
    );

    input.close();

    cout << "Encrypted file read successfully!" << endl;
    cout << "Bytes read: " << buffer.size() << endl;

    // Initialize RC4 S array
    unsigned char S[256];

    for (int i = 0; i < 256; i++) {
        S[i] = i;
    }

    // RC4 Key Scheduling Algorithm
    int j = 0;

    for (int i = 0; i < 256; i++) {
        j = (j + S[i] + key[i % key.length()]) % 256;

        unsigned char temp = S[i];
        S[i] = S[j];
        S[j] = temp;
    }

    // RC4 Pseudo-Random Generation Algorithm
    int i = 0;
    j = 0;

    for (auto& byte : buffer) {
        i = (i + 1) % 256;
        j = (j + S[i]) % 256;

        unsigned char temp = S[i];
        S[i] = S[j];
        S[j] = temp;

        unsigned char keystream = S[(S[i] + S[j]) % 256];
    }

```

```

    byte ^= keystream;
}

// Write decrypted data
ofstream output("file_decrypted.txt", ios::binary);

if (!output) {
    cerr << "Error creating file_decrypted.txt" << endl;
    return 1;
}

output.write(
    reinterpret_cast<const char*>(buffer.data()),
    buffer.size()
);

output.close();

cout << "File decrypted successfully!" << endl;

return 0;
}

```

```

// rc4_encryptor.cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <iterator>

using namespace std;

int main() {
    string key = "mysecretkey";
    unsigned char S[256];
    for (int i = 0; i < 256; i++) {
        S[i] = i;
    }
    int j = 0;
    for (int i = 0; i < 256; i++) {
        j = (j + S[i] + key[i % key.length()]) % 256;
        unsigned char temp = S[i];
        S[i] = S[j];
        S[j] = temp;
    }
    ifstream input("file.txt", ios::binary);
    if (!input) {
        cerr << "Error opening file.txt" << endl;
        return 1;
    }
    vector<unsigned char> buffer(
        (istreambuf_iterator<char>(input)),
        (istreambuf_iterator<char>())
    );
    input.close();
    cout << "File read successfully!" << endl;
    cout << "Bytes read: " << buffer.size() << endl;
    return 0;
}

```

```

~/Downloads/Telegram Desktop/malware
$ g++ rc4_encryptor.cpp -o rc4_encryptor
~/Downloads/Telegram Desktop/malware
$ ./rc4_encryptor
File read successfully!
Bytes read: 20
File encrypted successfully!

~/Downloads/Telegram Desktop/malware
$ ls -l file.txt file_encrypted.bin
-rw-rw-r-- 1 marpisc marpisc 20 25 16:28 file_encrypted.bin
-rw-rw-r-- 1 marpisc marpisc 20 19 14:13 file.txt
$ xxd file.txt
00000000: 736f 6d65 2066 696c 6520 746f 2065 6e63  some file to enc
00000010: 7279 7074                                     ryp
00000020: 99bc 15dd e02f 57fc a52d 4a52 da27 aadd  ...../W..-JR...'
00000030: 7b2c d56e                                     {,.n

```


RC4 Decryptor

After successfully creating and testing the encryptor, I created the second C++ file, called `rc4_decryptor.cpp`. The purpose of the decryptor is to take the encrypted binary file and restore its original contents.

The decryptor uses the same RC4 key that was used by the encryptor:

```
string key = "mysecretkey";
```

Using the same key is important because RC4 is a symmetric-key cipher. The decryptor needs to generate the same keystream that was generated during encryption.

Reading the Encrypted File

The decryptor first opens the encrypted file in binary mode:

```
ifstream input("file_encrypted.bin", ios::binary);
```

I used binary mode because the encrypted file contains arbitrary byte values and should not be treated as normal text.

The encrypted contents are then loaded into a memory buffer:

```
vector<unsigned char> buffer(  
    (istreambuf_iterator<char>(input)),  
    istreambuf_iterator<char>()  
);
```

This allows the program to process the encrypted data byte by byte.

The program also checks whether the file was opened successfully. If the file cannot be opened, it displays an error message and stops.

RC4 Key Scheduling

The decryptor then initializes the RC4 `S` array with values from 0 to 255, just like the encryptor.

The same Key Scheduling Algorithm (KSA) is then performed using the same key:

```

int j = 0;

for (int i = 0; i < 256; i++) {
    j = (j + S[i] + key[i % key.length()]) % 256;

    unsigned char temp = S[i];
    S[i] = S[j];
    S[j] = temp;
}

```

This is important because using the same key and the same initial state allows the decryptor to produce the same RC4 keystream as the encryptor.

Decryption Using PRGA

After the KSA, the decryptor uses the RC4 Pseudo-Random Generation Algorithm (PRGA).

For each byte in the encrypted buffer, the program generates a keystream byte and performs XOR:

```

unsigned char keystream = S[(S[i] + S[j]) % 256];

byte ^= keystream;

```

This restores the original data because XOR is reversible. The encryption process was:

Original data XOR keystream = encrypted data

The decryption process performs:

Encrypted data XOR same keystream = original data

Therefore, the same RC4 algorithm and key can be used to reverse the encryption.

Writing the Decrypted File

After processing all the encrypted bytes, the decryptor creates:

```
ofstream output("file_decrypted.txt", ios::binary);
```

The decrypted bytes stored in the buffer are then written to this file.

After the program finished running, `file_decrypted.txt` contained:

some file to encrypt

which was the same content as the original `file.txt`.

Verification

I verified the result in two ways.

First, I used the `cmp` command:

```
cmp file.txt file_decrypted.txt
```

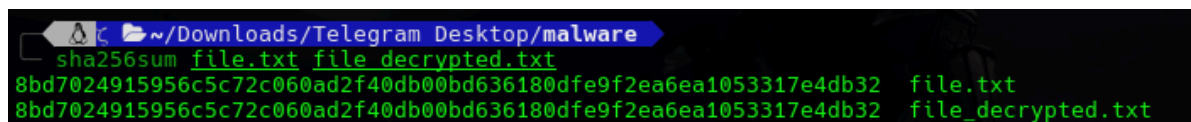
The command produced no output, meaning there was no difference between the two files.

I then used SHA-256 hashing to perform a second verification:

```
sha256sum file.txt file_decrypted.txt
```

The result was:

```
8bd7024915956c5c72c060ad2f40db00bd636180dfe9f2ea6ea1053317e4db32 file.txt
8bd7024915956c5c72c060ad2f40db00bd636180dfe9f2ea6ea1053317e4db32 file_decrypted.txt
```

A terminal window with a dark background and a blue title bar. The title bar text is '~ / Downloads / Telegram Desktop / malware'. The terminal shows the command 'sha256sum file.txt file_decrypted.txt' being executed. The output consists of two lines, each showing a long hexadecimal hash followed by the filename. Both files have the same hash: 8bd7024915956c5c72c060ad2f40db00bd636180dfe9f2ea6ea1053317e4db32.

```
~/Downloads/Telegram Desktop/malware
sha256sum file.txt file_decrypted.txt
8bd7024915956c5c72c060ad2f40db00bd636180dfe9f2ea6ea1053317e4db32 file.txt
8bd7024915956c5c72c060ad2f40db00bd636180dfe9f2ea6ea1053317e4db32 file_decrypted.txt
```

Both files produced exactly the same SHA-256 hash. This confirms that the decrypted file is identical to the original file, not just visually similar.

Final Result

The complete process was successfully tested:

file.txt
↓
RC4 encryption
↓
file_encrypted.bin
↓
RC4 decryption
↓
file_decrypted.txt

The encrypted file contained different binary data from the original, while the decrypted file was restored to its original contents.

The successful **cmp** comparison and matching SHA-256 hashes confirmed that the RC4 encryptor and decryptor worked correctly for the test file.

The final code review

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <iterator>

using namespace std;

int main() {
    string key = "mysecretkey";

    ifstream input("file_encrypted.bin", ios::binary);

    if (!input) {
        cerr << "Error opening file_encrypted.bin" << endl;
        return 1;
    }

    vector<unsigned char> buffer(
        (istreambuf_iterator<char>(input)),
        istreambuf_iterator<char>());

    input.close();

    cout << "Encrypted file read successfully!" << endl;
    cout << "Bytes read: " << buffer.size() << endl;

    unsigned char S[256];

    for (int i = 0; i < 256; i++) {
```

```

        S[i] = i;
    }

    int j = 0;

    for (int i = 0; i < 256; i++) {
        j = (j + S[i] + key[i % key.length()]) % 256;

        unsigned char temp = S[i];
        S[i] = S[j];
        S[j] = temp;
    }

    int i = 0;
    j = 0;

    for (auto& byte : buffer) {
        i = (i + 1) % 256;
        j = (j + S[i]) % 256;

        unsigned char temp = S[i];
        S[i] = S[j];
        S[j] = temp;

        unsigned char keystream = S[(S[i] + S[j]) % 256];

        byte ^= keystream;
    }

    ofstream output("file_decrypted.txt", ios::binary);

    if (!output) {
        cerr << "Error creating file_decrypted.txt" << endl;
        return 1;
    }

    output.write(
        reinterpret_cast<const char*>(buffer.data()),
        buffer.size()
    );

    output.close();

    cout << "File decrypted successfully!" << endl;

    return 0;
}

```